

1. Clase ProductCatalog

```
public class ProductCatalog {

    private final int[] ids;
    private final String[] names;
    private final double[] prices;
    private final int[] stock;

    private boolean idsOrdenados;

    public ProductCatalog(int[] ids, String[] names, double[] prices,
int[] stock) {
        if (ids == null || names == null || prices == null || stock ==
null) {
            throw new RuntimeException("Los arrays del catálogo no pueden
ser null.");
        }
        int n = ids.length;
        if (n == 0) {
            throw new RuntimeException("El catálogo no puede estar
vacío.");
        }
        if (names.length != n || prices.length != n || stock.length != n)
{
            throw new RuntimeException("Los arrays del catálogo deben
tener la misma longitud.");
        }

        this.ids = ids;
        this.names = names;
        this.prices = prices;
        this.stock = stock;
        this.idsOrdenados = false;
    }

    public void printCatalog() {
        System.out.println("\n📖 CATÁLOGO");
        System.out.println("-----");
        System.out.println("-----");
        System.out.printf("%-6s %-12s %-10s %-6s\n", "ID", "Nombre",
"Precio", "Stock");
        System.out.println("-----");
        System.out.println("-----");
        for (int i = 0; i < ids.length; i++) {
            System.out.printf("%-6d %-12s %-10.2f %-6d\n", ids[i],
names[i], prices[i], stock[i]);
        }
    }
}
```

```

    }
    System.out.println("-----\n");
}

public int indexOfId(int idBuscado) {
    for (int i = 0; i < ids.length; i++) {
        if (ids[i] == idBuscado) return i;
    }
    return -1;
}

public int indexOfNameContains(String text) {
    if (text == null) return -1;
    String t = text.trim().toLowerCase();
    for (int i = 0; i < names.length; i++) {
        if (names[i] != null && names[i].toLowerCase().contains(t))
return i;
    }
    return -1;
}

public void bubbleSortArrays() {
    int n = ids.length;
    boolean swapped;
    for (int pass = 0; pass < n - 1; pass++) {
        swapped = false;
        for (int i = 0; i < n - 1 - pass; i++) {
            if (ids[i] > ids[i + 1]) {
                swapAll(i, i + 1);
                swapped = true;
            }
        }
        if (!swapped) break;
    }
    idsOrdenados = true;
}

public int binarySearchId(int idBuscado, boolean idsOrdenadosParam) {
    if (!idsOrdenadosParam) return -1;

    int left = 0;
    int right = ids.length - 1;

    while (left <= right) {
        int mid = (left + right) / 2;
        if (ids[mid] == idBuscado) return mid;
        if (ids[mid] < idBuscado) left = mid + 1;
        else right = mid - 1;
    }
}

```

```

    }
    return -1;
}

public double getPriceByIndex(int idx) {
    checkIndex(idx);
    return prices[idx];
}

public int getStockByIndex(int idx) {
    checkIndex(idx);
    return stock[idx];
}

public int size() {
    return ids.length;
}

public String getNameByIndex(int idx) {
    checkIndex(idx);
    return names[idx];
}

public int getIdByIndex(int idx) {
    checkIndex(idx);
    return ids[idx];
}

public void bubbleSort() {
    bubbleSortArrays();
}

public int binarySearchId(int idBuscado) {
    return binarySearchId(idBuscado, idsOrdenados);
}

public void decreaseStock(int idx, int qty) {
    checkIndex(idx);
    if (qty <= 0) throw new RuntimeException("La cantidad a restar debe ser > 0.");
    if (stock[idx] < qty) throw new RuntimeException("Stock insuficiente para esa cantidad.");
    stock[idx] -= qty;
}

public void printProductDetail(int idx) {
    checkIndex(idx);
    System.out.println("\n❑ FICHA PRODUCTO");
}

```

```

        System.out.println("-----");
        System.out.println("ID:      " + ids[idx]);
        System.out.println("Nombre: " + names[idx]);
        System.out.printf("Precio: %.2f €%n", prices[idx]);
        System.out.println("Stock:  " + stock[idx]);
        System.out.println("-----");
    }

    public boolean isIdsOrdenados() {
        return idsOrdenados;
    }

    private void checkIndex(int idx) {
        if (idx < 0 || idx >= ids.length) {
            throw new RuntimeException("Índice fuera de rango: " + idx);
        }
    }

    private void swapAll(int i, int j) {
        int tmpId = ids[i];
        ids[i] = ids[j];
        ids[j] = tmpId;

        String tmpName = names[i];
        names[i] = names[j];
        names[j] = tmpName;

        double tmpPrice = prices[i];
        prices[i] = prices[j];
        prices[j] = tmpPrice;

        int tmpStock = stock[i];
        stock[i] = stock[j];
        stock[j] = tmpStock;
    }
}

```

2. Clase Cart

```

public class Cart {

    private final int[] cartIds;
    private final int[] cartQty;
    private int cartSize;
}

```

```

public Cart() {
    this.cartIds = new int[10];
    this.cartQty = new int[10];
    this.cartSize = 0;
}

public void printCart(ProductCatalog catalog) {
    if (isEmpty()) {
        System.out.println("\n CARRITO VACÍO\n");
        return;
    }

    System.out.println("\n CARRITO");
    System.out.println("-----");
    System.out.printf("%-12s %-6s %-10s\n", "Producto", "Cant",
"Subtotal");
    System.out.println("-----");

    double total = 0.0;

    for (int i = 0; i < cartSize; i++) {
        int id = cartIds[i];
        int qty = cartQty[i];

        int idxCatalog = catalog.indexOfId(id);
        if (idxCatalog == -1) {
            System.out.printf("%-12s %-6d %-10s\n", "(ID " + id +
")", qty, "N/A");
            continue;
        }

        double price = catalog.getPriceByIndex(idxCatalog);
        double subtotal = qty * price;
        total += subtotal;

        String name = catalog.getNameByIndex(idxCatalog);

        System.out.printf("%-12s %-6d %-10.2f\n", name, qty,
subtotal);
    }

    System.out.println("-----");
    System.out.printf("TOTAL: %.2f €\n", total);
    System.out.println("-----\n");
}

```

```

    }

    public int indexOfCartId(int id) {
        for (int i = 0; i < cartSize; i++) {
            if (cartIds[i] == id) return i;
        }
        return -1;
    }

    public void add(int id, int qty, ProductCatalog catalog) {
        if (qty <= 0) throw new RuntimeException("La cantidad debe ser > 0.");

        int idxCatalog = catalog.indexOfId(id);
        if (idxCatalog == -1) throw new RuntimeException("El ID no existe en el catálogo.");

        int availableStock = catalog.getStockByIndex(idxCatalog);

        int idxCart = indexOfCartId(id);

        if (idxCart != -1) {
            int newQty = cartQty[idxCart] + qty;
            if (newQty > availableStock) {
                throw new RuntimeException("No hay stock suficiente. Stock disponible: " + availableStock);
            }
            cartQty[idxCart] = newQty;
            return;
        }

        if (cartSize >= cartIds.length) {
            throw new RuntimeException("El carrito está lleno (máximo 10 líneas).");
        }
        if (qty > availableStock) {
            throw new RuntimeException("No hay stock suficiente. Stock disponible: " + availableStock);
        }

        cartIds[cartSize] = id;
        cartQty[cartSize] = qty;
        cartSize++;
    }

    public void remove(int id) {
        int idx = indexOfCartId(id);
        if (idx == -1) {
            System.out.println("⚠ Ese ID no está en el carrito.");
        }
    }

```

```

        return;
    }

    for (int i = idx; i < cartSize - 1; i++) {
        cartIds[i] = cartIds[i + 1];
        cartQty[i] = cartQty[i + 1];
    }

    cartIds[cartSize - 1] = 0;
    cartQty[cartSize - 1] = 0;

    cartSize--;
}

public void clear() {
    cartSize = 0; // con esto basta
}

public boolean isEmpty() {
    return cartSize == 0;
}

public int getCartSize() {
    return cartSize;
}

public int getCartIdAt(int i) {
    if (i < 0 || i >= cartSize) throw new RuntimeException("Índice de
carrito fuera de rango.");
    return cartIds[i];
}

public int getCartQtyAt(int i) {
    if (i < 0 || i >= cartSize) throw new RuntimeException("Índice de
carrito fuera de rango.");
    return cartQty[i];
}
}

```

3. Clase SalesLog

```

public class SalesLog {

    private final int[][] sales;
    private int salesSize;
}

```

```

public SalesLog(int capacity) {
    if (capacity <= 0) {
        throw new RuntimeException("La capacidad de ventas debe ser >
0.");
    }
    this.sales = new int[capacity][2];
    this.salesSize = 0;
}

public void addSale(int productId, int qty) {
    if (qty <= 0) throw new RuntimeException("La cantidad vendida
debe ser > 0.");
    if (salesSize >= sales.length) {
        throw new RuntimeException("No se pueden registrar más ventas
(SalesLog lleno).");
    }
    sales[salesSize][0] = productId;
    sales[salesSize][1] = qty;
    salesSize++;
}

public void printSalesTable(ProductCatalog catalog) {
    if (salesSize == 0) {
        System.out.println("\n[ ] No hay ventas registradas.\n");
        return;
    }

    System.out.println("\n[ ] TABLA DE VENTAS");
    System.out.println("-----
-----");
    System.out.printf("%-6s %-6s %-12s %-6s\n", "#", "ID", "Nombre",
"Qty");
    System.out.println("-----
-----");

    for (int i = 0; i < salesSize; i++) {
        int id = sales[i][0];
        int qty = sales[i][1];

        int idxCatalog = catalog.indexOfId(id);
        String name = (idxCatalog == -1) ? "(desconocido)" :
catalog.getNameByIndex(idxCatalog);

        System.out.printf("%-6d %-6d %-12s %-6d\n", i, id, name,
qty);
    }

    System.out.println("-----
-----\n");
}

```

```

    }

    public int[] computeUnitsSoldPerProduct(ProductCatalog catalog) {
        int n = catalog.size(); // tamaño del catálogo
        int[] soldUnits = new int[n];

        for (int i = 0; i < salesSize; i++) {
            int id = sales[i][0];
            int qty = sales[i][1];

            int idxCatalog = catalog.indexOfId(id);
            if (idxCatalog != -1) {
                soldUnits[idxCatalog] += qty;
            }
        }

        return soldUnits;
    }

    public void printTopSeller(ProductCatalog catalog) {
        if (salesSize == 0) {
            System.out.println("Σ Top seller: (sin ventas todavía)\n");
            return;
        }

        int[] soldUnits = computeUnitsSoldPerProduct(catalog);

        int bestIdx = -1;
        int bestUnits = -1;

        for (int i = 0; i < soldUnits.length; i++) {
            if (soldUnits[i] > bestUnits) {
                bestUnits = soldUnits[i];
                bestIdx = i;
            }
        }

        if (bestIdx == -1 || bestUnits <= 0) {
            System.out.println("Σ Top seller: (no hay unidades vendidas registradas)\n");
            return;
        }

        System.out.println("Σ TOP SELLER");
        System.out.println("-----");
        System.out.println("-----");
        System.out.println("Producto: " +
catalog.getNameByIndex(bestIdx));
        System.out.println("ID: " + catalog.getIdByIndex(bestIdx));
    }

```

```

        System.out.println("Unidades: " + bestUnits);
        System.out.println("-----\n");
    }
}

```

4. Clase OrderService

```

public class OrderService {

    public void confirmOrder(Cart cart, ProductCatalog catalog, SalesLog
salesLog) {

        if (cart.isEmpty()) {
            System.out.println("⚠ El carrito está vacío. No hay pedido
que confirmar.\n");
            return;
        }

        System.out.println("\n📄 TICKET DE COMPRA");
        System.out.println("-----");
        System.out.printf("%-12s %-6s %-10s\n", "Producto", "Cant",
"Subtotal");
        System.out.println("-----");

        double total = 0.0;

        for (int i = 0; i < cart.getCartSize(); i++) {

            int id = cart.getCartIdAt(i);
            int qty = cart.getCartQtyAt(i);

            int idxCatalog = catalog.indexOfId(id);
            if (idxCatalog == -1) {
                throw new RuntimeException("Producto inexistente en
catálogo: " + id);
            }

            double price = catalog.getPriceByIndex(idxCatalog);
            double subtotal = price * qty;
            total += subtotal;

            catalog.decreaseStock(idxCatalog, qty);
        }
    }
}

```

```

        salesLog.addSale(id, qty);

        System.out.printf("%-12s %-6d %-10.2f%n",
            catalog.getNameByIndex(idxCatalog),
            qty,
            subtotal);
    }

    System.out.println("-----");
    System.out.printf("TOTAL: %.2f €\n", total);
    System.out.println("-----\n");

    cart.clear();
}
}

```

5. Clase MiniMarketApp

```

import java.util.Scanner;

public class MiniMarketApp {
    public static void main(String[] args) {
        int[] ids = {101, 205, 150, 310, 404, 120, 333};
        String[] names = {"Pan", "Leche", "Huevos", "Pasta", "Queso", "Arroz",
            "Tomate"};
        double[] prices = {1.20, 1.05, 2.90, 1.30, 3.75, 1.10, 0.95};
        int[] stock = {20, 15, 8, 25, 6, 30, 18};
        ProductCatalog catalog = new ProductCatalog(ids, names, prices, stock);
        Cart cart = new Cart();
        SalesLog salesLog = new SalesLog(20);
        OrderService orderService = new OrderService();
        Scanner sc = new Scanner(System.in);
        int option;
        do {
            printMenu();
            option = readInt(sc, "Elige opción: ");
            switch (option) {
                case 1 -> catalog.printCatalog();
                case 2 -> searchMenu(sc, catalog);
                case 3 -> {
                    int id = readInt(sc, "ID producto: ");
                    int qty = readInt(sc, "Cantidad: ");
                    try {
                        cart.add(id, qty, catalog);
                    }
                }
            }
        } while (option != 0);
    }
}

```

```

System.out.println("✅ Añadido al carrito.\n");
} catch (RuntimeException ex) {
System.out.println("❌ " + ex.getMessage() + "\n");
}
}
case 4 -> cart.printCart(catalog);
case 5 -> {
int id = readInt(sc, "ID a eliminar del carrito: ");
cart.remove(id);
System.out.println();
}
case 6 -> {
try {
orderService.confirmOrder(cart, catalog, salesLog);
} catch (RuntimeException ex) {
System.out.println("❌ " + ex.getMessage() + "\n");
}
}
case 7 -> {
salesLog.printSalesTable(catalog);
salesLog.printTopSeller(catalog);
}
case 8 -> System.out.println("👋 Saliendo...");
default -> System.out.println("⚠️ Opción inválida.\n");
}
} while (option != 8);
sc.close();
}

private static void printMenu() {
System.out.println("🛒 MINI-MARKET");
System.out.println("1) 📖 Ver catálogo");
System.out.println("2) 🔍 Buscar producto");
System.out.println("3) 🛒 Añadir al carrito");
System.out.println("4) 🛒 Ver carrito");
System.out.println("5) ❌ Eliminar del carrito");
System.out.println("6) ✅ Confirmar pedido");
System.out.println("7) 📊 Ventas por tickets");
System.out.println("8) 🚪 Salir");
System.out.println("-----");
}

private static void searchMenu(Scanner sc, ProductCatalog catalog) {
System.out.println("\n🔍 BÚSQUEDA");
System.out.println("1) Por ID (lineal)");
System.out.println("2) Por nombre (contains)");
System.out.println("3) Por ID (binaria, se ordenado el array por ID)");
int op = readInt(sc, "Elige opción de búsqueda: ");
switch (op) {
case 1 -> {

```

```

int id = readInt(sc, "ID a buscar: ");
int idx = catalog.indexOfId(id);
if (idx == -1) System.out.println("✗ No encontrado.\n");
else catalog.printProductDetail(idx);
}
case 2 -> {
System.out.print("Texto a buscar: ");
String text = sc.nextLine();
int idx = catalog.indexOfNameContains(text);
if (idx == -1) System.out.println("✗ No encontrado.\n");
else catalog.printProductDetail(idx);
}
case 3 -> {
int id = readInt(sc, "ID a buscar: ");
catalog.bubbleSort();
int idx = catalog.binarySearchId(id);
if (idx == -1) {
System.out.println("✗ No encontrado.\n");
} else {
catalog.printProductDetail(idx);
}
}
default -> System.out.println("⚠ Opción inválida.\n");
}
}
private static int readInt(Scanner sc, String prompt) {
while (true) {
System.out.print(prompt);
String line = sc.nextLine().trim();
try {
return Integer.parseInt(line);
} catch (NumberFormatException ex) {
System.out.println("⚠ Debes introducir un número entero.");
}
}
}
}
}

```